Technical Requirements Document (TRD): AuraFlow

Project Name: FluxHeal — Autonomous Self-Healing ETL/ELT Data Pipeline

Architecture Stack: NestJS (TypeScript, Node.js) + PostgreSQL (Relational + `JSONB` Hybrid Storage)

Real-Time Transport: NestJS `@WebSocketGateway` (Socket.io) + EventEmitter2

Document Version: v2.0 (Engineering Execution Specification)

---

1. System Architecture & Module Boundaries

The backend is built as a modular NestJS monolith, decoupling high-throughput ingestion from asynchronous anomaly quarantine, AI diagnosis, and PostgreSQL transactional savepoint patches.

```
src/
├── main.ts
├── app.module.ts
├── common/
│   ├── interfaces/          # Shared TypeScript contracts (Telemetry, Rule, Anomaly)
│   └── filters/             # PgExceptionFilter (Intercepts Postgres SQL error codes)
├── modules/
│   ├── ingestion/           # Fast-path REST & WebSocket stream receivers
│   ├── validator/           # In-memory Schema Gate + Hot-Rule Cache (<1ms)
│   ├── quarantine/          # Dead-Letter Queue (DLQ) manager & PostgreSQL persistence
│   ├── healer/              # Tier-1 Deterministic Imputer + Tier-2 LLM Diagnostic Agent
│   ├── elt-warehouse/       # Raw JSONB staging, SQL runner & Savepoint Dry-Run Engine
│   ├── green-metrics/       # Cloud CPU, kWh & Carbon (gCO2e) savings calculator
│   ├── chaos-simulator/     # One-click fault injector for live hackathon
```

```
demos
|   └── gateway/              # @WebSocketGateway pushing 60fps state to
React/Canvas UI
```

Core In-Memory Accelerators

1. `HotRuleCacheService` **(** `@Injectable()` **Singleton):**

- Hydrates active rules from the PostgreSQL `healing_rules_registry` table on `OnModuleInit()`.
- Stores rules in an in-memory `Map<string, CompiledHealingRule>` keyed by `source_signature`.
- Executes learned schema mappings and regex unit-strippers in **$<0.5\text{ms}$** before validation fails.

2. `TimeSeriesWindowService` **(** `@Injectable()` **Singleton):**

- Maintains an in-memory circular buffer (`Map<string, number[]>`) holding the last 20 valid readings per `device_id`.
- Imputes `null`, `NaN`, or extreme spikes (`-9999`) using **Exponential Moving Average (EMA)** or linear interpolation in microseconds.

---

2. PostgreSQL Database Schema & Storage Engine (DDL)

PostgreSQL serves as the Relational Warehouse, Raw `JSONB` Staging Lake, and Quarantine/Rule Registry.

```
-- Enable UUID generation
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- 1. ENUMS FOR STRICT STATE MANAGEMENT
CREATE TYPE anomaly_category AS ENUM (
    'SCHEMA_DRIFT',
    'SENSOR_OUTLIER',
    'TYPE_MISMATCH',
    'ELT_SQL_ERROR',
    'API_RATE_LIMIT'
);
```

```
CREATE TYPE quarantine_status AS ENUM (
    'ISOLATED',
    'DIAGNOSING',
    'AUTO_HEALED',
    'PENDING_HITL',
    'REVERTED'
);

-- 2. TARGET PRODUCTION WAREHOUSE
CREATE TABLE clean_telemetry_warehouse (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    device_id VARCHAR(64) NOT NULL,
    metric_timestamp TIMESTAMPTZ NOT NULL,
    power_kw DOUBLE PRECISION NOT NULL,
    voltage_v DOUBLE PRECISION,
    temperature_c DOUBLE PRECISION,
    moisture_pct DOUBLE PRECISION,
    is_healed BOOLEAN DEFAULT FALSE,
    _healed_meta JSONB DEFAULT NULL,
    ingested_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_warehouse_device_ts ON clean_telemetry_warehouse (device
CREATE INDEX idx_warehouse_healed_meta ON clean_telemetry_warehouse USING

-- 3. RAW ELT STAGING TABLE
CREATE TABLE raw_staging_payloads (
    batch_id UUID NOT NULL,
    row_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_connector VARCHAR(64) NOT NULL,
    raw_payload JSONB NOT NULL,
    processed BOOLEAN DEFAULT FALSE,
    received_at TIMESTAMPTZ DEFAULT NOW()
);

-- 4. QUARANTINE DEAD-LETTER QUEUE (DLQ)
CREATE TABLE quarantine_dlq (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_connector VARCHAR(64) NOT NULL,
    raw_payload JSONB NOT NULL,
    healed_payload JSONB DEFAULT NULL,
    anomaly_type anomaly_category NOT NULL,
    error_message TEXT NOT NULL,
    status quarantine_status DEFAULT 'ISOLATED',
```

```
    confidence_score NUMERIC(4,3),
    applied_rule_id UUID,
    quarantined_at TIMESTAMPTZ DEFAULT NOW(),
    resolved_at TIMESTAMPTZ
);

-- 5. AUTONOMOUS HEALING RULES REGISTRY
CREATE TABLE healing_rules_registry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_signature VARCHAR(255) UNIQUE NOT NULL,
    anomaly_category anomaly_category NOT NULL,
    rule_definition JSONB NOT NULL,
    sql_patch_template TEXT DEFAULT NULL,
    confidence_score NUMERIC(4,3) NOT NULL,
    times_triggered INTEGER DEFAULT 1,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 6. GREEN COMPUTE & CARBON SAVINGS LEDGER
CREATE TABLE green_compute_ledger (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_id UUID NOT NULL,
    pipeline_mode VARCHAR(10) NOT NULL,
    total_batch_rows INTEGER NOT NULL,
    quarantined_rows INTEGER NOT NULL,
    avoided_backfill_rows INTEGER NOT NULL,
    cpu_ms_saved DOUBLE PRECISION NOT NULL,
    energy_wh_saved DOUBLE PRECISION NOT NULL,
    co2_grams_saved DOUBLE PRECISION NOT NULL,
    recorded_at TIMESTAMPTZ DEFAULT NOW()
);
```

---

3. NestJS Core Service Implementation

3.1 Real-Time ETL Stream Ingestion (`IngestionService`)

```
import { Injectable } from '@nestjs/common';
import { EventEmitter2 } from '@nestjs/event-emitter';

@Injectable()
```

```
export class IngestionService {
  constructor(
    private readonly hotCache: HotRuleCacheService,
    private readonly validator: ValidatorService,
    private readonly warehouseRepo: WarehouseRepository,
    private readonly eventEmitter: EventEmitter2,
    private readonly wsGateway: PipelineGateway,
  ) {}

  async processIncomingStream(source: string, payload: Record<string, any
    const signature = this.hotCache.computeSignature(source, payload);
    const cachedRule = this.hotCache.getRule(signature);

    let workingPayload = cachedRule
      ? this.hotCache.applyDeterministicTransform(payload, cachedRule)
      : payload;

    const validation = this.validator.validateTelemetry(workingPayload);

    if (validation.isValid) {
      await this.warehouseRepo.insertCleanRecord(validation.cleanRecord);
      this.wsGateway.emitFastPathPacket(validation.cleanRecord);
      return { status: 'FAST_PATH_OK', ruleApplied: cachedRule?.id ?? nul
    }

    this.eventEmitter.emit('pipeline.anomaly.detected', {
      source,
      signature,
      rawPayload: payload,
      errors: validation.errors,
    });

    return { status: 'QUARANTINED_FOR_HEALING', errors: validation.errors
  }
}
```

3.2 Transactional ELT Warehouse SQL Patching (`EltWarehouseService`)

```
@Injectable()
export class EltWarehouseService {
  constructor(
```

```
    private readonly dataSource: DataSource,
    private readonly healerService: HealerService,
    private readonly greenMetricsService: GreenMetricsService,
  ) {}

  async executeAndHealEltQuery(batchId: string, brokenSql: string): Promi
    const queryRunner = this.dataSource.createQueryRunner();
    await queryRunner.connect();
    await queryRunner.startTransaction();

    try {
      await queryRunner.query('SAVEPOINT elt_execution');
      await queryRunner.query(brokenSql, [batchId]);
      await queryRunner.commitTransaction();
    } catch (pgError: any) {
      await queryRunner.query('ROLLBACK TO SAVEPOINT elt_execution');

      const schemaContext = await this.introspectStagingSchema(queryRunne
      const patchedPlan = await this.healerService.patchSqlQuery(
        brokenSql,
        pgError.code,
        pgError.message,
        schemaContext,
      );

      await queryRunner.query('SAVEPOINT elt_healed_retry');
      await queryRunner.query(patchedPlan.patchedSql, [batchId]);
      await queryRunner.commitTransaction();

      await this.greenMetricsService.recordAvoidedBackfill(batchId);
    } finally {
      await queryRunner.release();
    }
  }
}
```

---

4. API Endpoints & WebSocket Contracts

4.1 REST API Controller Endpoints

Method	Endpoint	Description
`POST`	`/api/v1/ingest/:connector`	Ingests single telemetry object or bulk array batches.
`POST`	`/api/v1/chaos/inject`	Triggers live fault simulations (*SCHEMA_DRIFT*, *SENSOR_OUTLIER*, *TYPE_MISMATCH*, *ELT_SQL_ERROR*).
`GET`	`/api/v1/dashboard/bootstrap`	Loads initial dashboard state, KPIs, and active rules.
`POST`	`/api/v1/quarantine/:id/approve`	Human-in-the-Loop (HITL) endpoint for low-confidence fixes (<90%).
`POST`	`/api/v1/rules/:id/revert`	Governance rollback: evicts rule from hot cache and restores raw values.

4.2 WebSocket Event Contract (`@WebSocketGateway('/stream')`)

Event Name	Direction	Payload Structure
`stream:packet`	Server ➔ UI	`{ id, status: 'FAST_PATH'
`anomaly:quarantined`	Server ➔ UI	`{ quarantineId, anomalyType, rawPayload, error }`
`healer:diagnosed`	Server ➔ UI	`{ quarantineId, patchedPayload, ruleDef, confidence }`
`rule:cached`	Server ➔ UI	`{ ruleId, signature, targetMapping }`
`green:updated`	Server ➔ UI	`{ rowsAvoided, cpuHoursSaved, co2GramsSaved }`

---

5. Green Compute & Carbon Savings Mathematical Formulation

1. **Avoided Re-Computation Rows (R_{saved}):**

$$R_{\text{saved}} = R_{\text{total_batch}} - R_{\text{quarantined}}$$

2. **Saved Server Compute Energy (E_{Wh}):**

$$E_{\text{Wh}} = \left(\frac{T_{\text{saved_cpu_ms}}}{3,600,000} \right) \times P_{\text{vCPU_watts}} \times \text{PUE}$$

(Where $P_{\text{vCPU_watts}} \approx 12\text{W}$ and $\text{PUE} = 1.2$).

3. **Prevented Carbon Emissions (C_{gCO2e}):**

$$C_{\text{gCO2e}} = \left(\frac{E_{\text{Wh}}}{1000} \right) \times I_{\text{grid}}$$

(Where $I_{\text{grid}} \approx 475\text{ gCO}_2/\text{kWh}$).